Density Estimation(KDE), EM Algorithm with MNIST Dataset

Qi(Sophie) Pan

1. Density estimation: Psychological experiments

In *Kanai, R., Feilden, T., Firth, C. and Rees, G., 2011. Political orientations are correlated with brain structure in young adults. Current biology, 21(8), pp.677-680.*, data are collected to study whether or not the two brain regions are likely to be independent of each other and considering different types of political view. The data set n90pol.csv contains information on 90 university students who participated in a psychological experiment designed to look for relationships between the size of different regions of the brain and political views. The variables amygdala and acc indicate the volume of two particular brain regions known to be involved in emotions and decision-making, the amygdala and the anterior cingulate cortex; more exactly, these are residuals from the predicted volume, after adjusting for height, sex, and similar body-type variables. The variable orientation gives the students' locations on a five-point scale from 1 (very conservative) to 5 (very liberal). Note that in the dataset, we only have observations for orientation from 2 to 5.

Recall in this case, the kernel density estimator (KDE) for a density is given by

$$p(x) = \frac{1}{m}\sum_{i=1}^{m}\frac{1}{h}K\left(\frac{x^i - x}{h}\right),$$

where x^i are two-dimensional vectors, $h > 0$ is the kernel bandwidth, based on the criterion we discussed in lecture. For one-dimensional KDE, use a one-dimensional Gaussian kernel

$$K(x) = \frac{1}{\sqrt{2\pi}}e^{-x^2/2}.$$

For two-dimensional KDE, use a two-dimensional Gaussian kernel: for

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2,$$

where x_1 and x_2 are the two dimensions respectively

$$K(x) = \frac{1}{2\pi}e^{-\frac{(x_1)^2+(x_2)^2}{2}}.$$

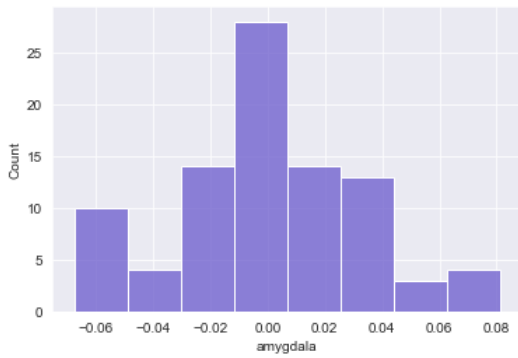
(a) Form the 1-dimensional histogram and KDE to estimate the distributions of amygdala and acc, respectively. For this question, you can ignore the variable orientation. Decide on a suitable number of bins so you can see the shape of the distribution clearly. Set an appropriate kernel bandwidth $h > 0$.

Answer:

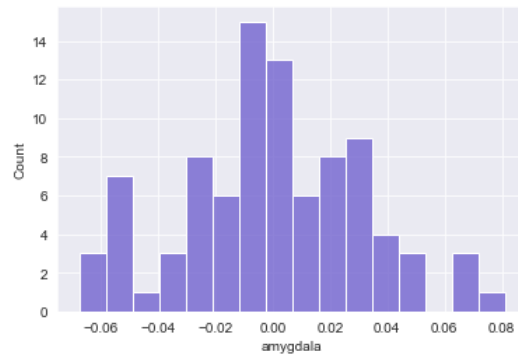
Histogram for the amygdala:

To form the 1-dimensional histogram for the amygdala, I have tried 4 different bin numbers, the default bin number is 8. I personally prefer bins =16 and 20. Either one shows the general distribution with enough details, but not too noisy like bin=25.

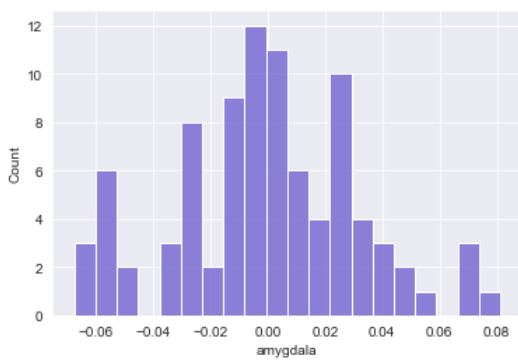
We can tell the modal of the distribution is around 0.00 and there is a small spike around -0.06 on the negative side.



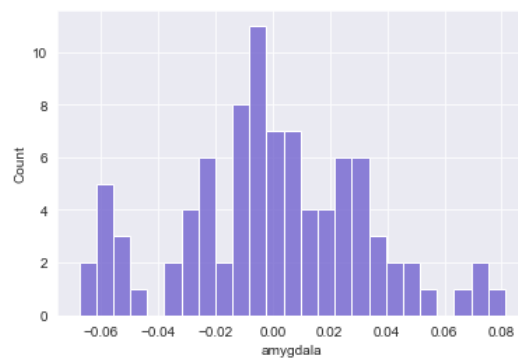
bin=8 (default)



bin=16



bin=20

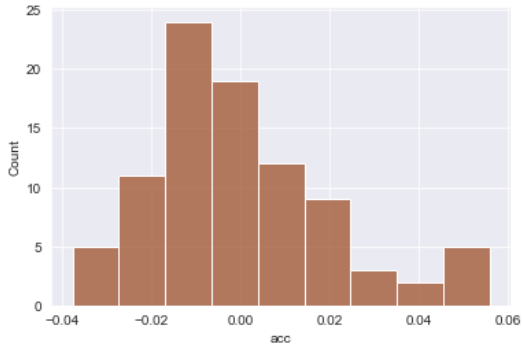


bin=25

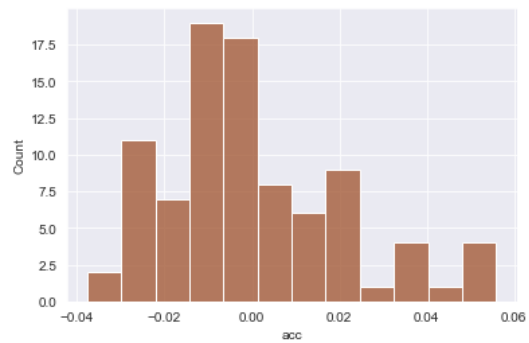
Histogram for the acc:

The same approach has been used for the 1-dimensional histogram of acc.

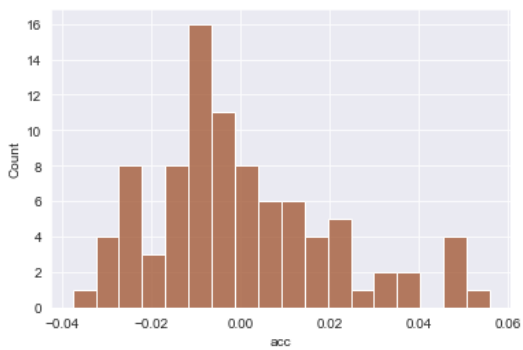
I prefer bin=18 for acc, which is just the right balance between overfitting the data and squeezing too much data within a few bins (losing the real structure). We can see the distribution is right skewed. The modal is around -0.01.



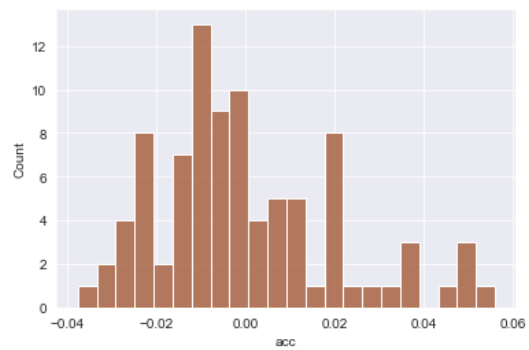
bin=9 (default)



bin=12



bin=18



bin=22

Some notes about the bandwidth formulation:

Based on my research and calculation, the "Silverman's rule of thumb" for bandwidth in our lecture and the formulation used in the Python packages Seaborn and Scipy are different. In addition, Seaborn and Scipy share the same method of bandwidth calculation.

In our lecture, the theoretical best bandwidth for 1-dimensional data is:

$$h \approx 1.06 * \hat{\sigma} * m^{-1/5}$$

h=the bandwidth, $\hat{\sigma}$=the std of our data, m=the number of our data points. Take the amygdala as an example:

$$\text{best h in lecture} = 1.06 * \hat{\sigma} * 90^{-1/5} = 0.014$$

Without $\hat{\sigma}$, we have:

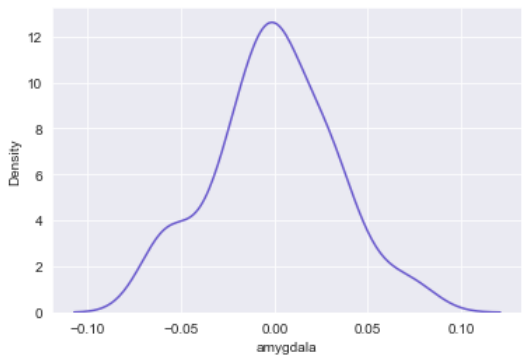
$$\text{best h in lecture no sigma} = 1.06 * 90^{-1/5} = 0.43$$

However, in Seaborn and Scipy packages, they just ignore the std. With d=the dimension of the data, we end with:

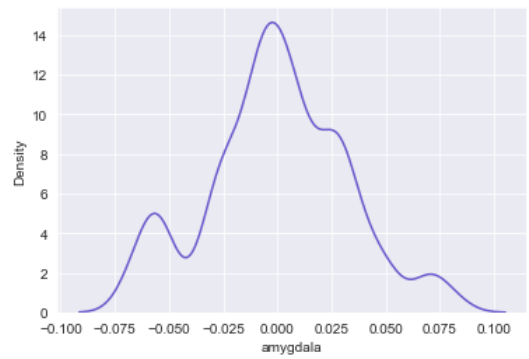
$$h = (m * (d+2)/4)^{(-1/(d+4))} = (90 * 0.75)^{-1/5} = 0.43$$

From above we can tell although the bandwidth formulation in coding packages is different from the one in our lecture, they are still aligned. Since Seaborn provides a very nice plotting result, my bandwidths below are under Seaborn's formulation rule.

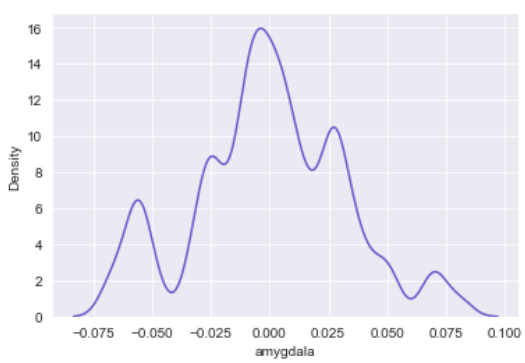
KDE for the amygdala:



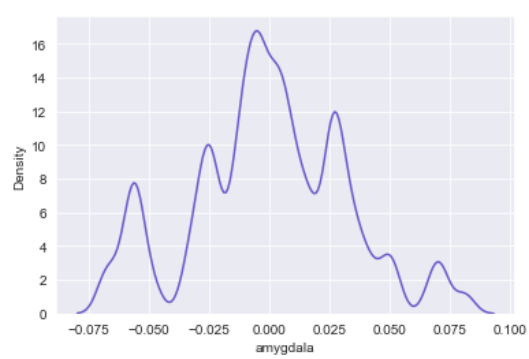
bandwidth=default



bandwidth=0.6



bandwidth=0.4

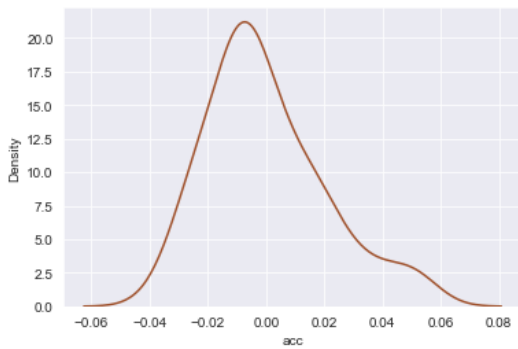


bandwidth=0.3

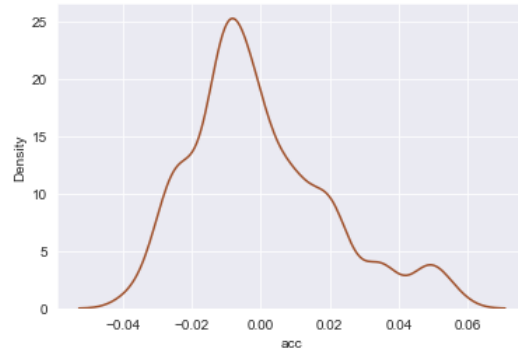
Although the best bandwidth from "Silverman's rule of thumb" is 0.43, it also depends on the data. We should find a balance between following the noise of the particular sample of data (overfitting) and smoothing the data so much that any real structure of the distribution is covered up (introducing bias). Based on these plots, I prefer bandwidth=0.6 since it is not that noisy but we can also see the general structure.

It is also aligned with the histogram that the modal is around 0 and there is a small peak on the negative side.

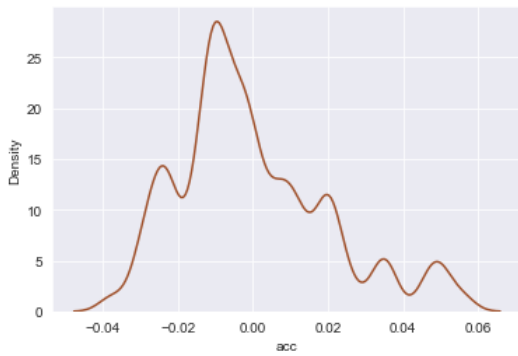
KDE for the acc:



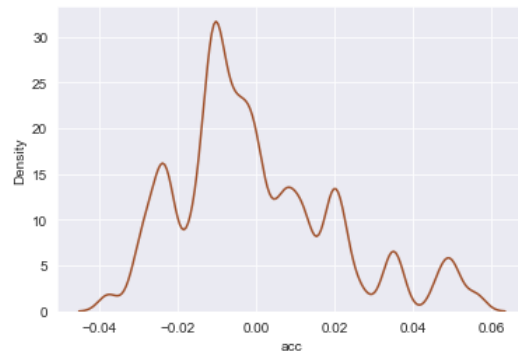
bandwidth=default



bandwidth=0.6



bandwidth=0.4

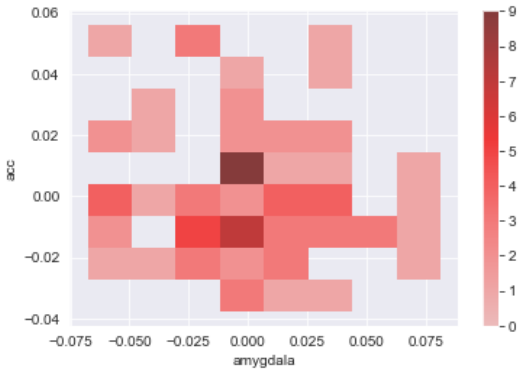


bandwidth=0.3

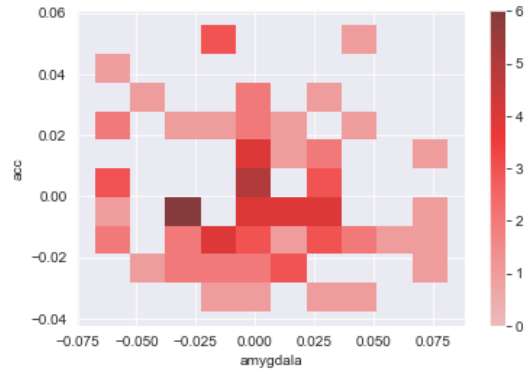
For acc, either the bandwidth=default or bandwidth=0.6 works. We can see the distribution is right skewed. The modal is around -0.01.

(b) Form 2-dimensional histogram for the pairs of variables (amygdala, acc). Decide on a suitable number of bins so you can see the shape of the distribution clearly.

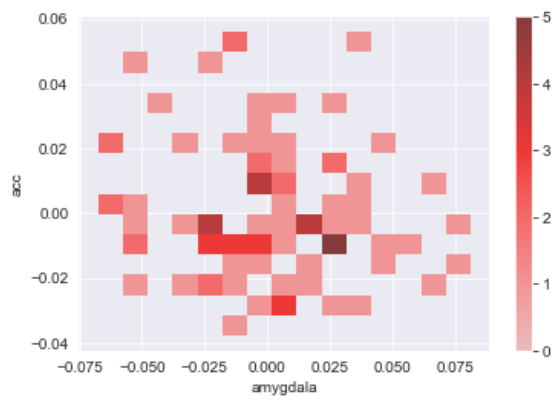
Answer:



bin=default



bin=10



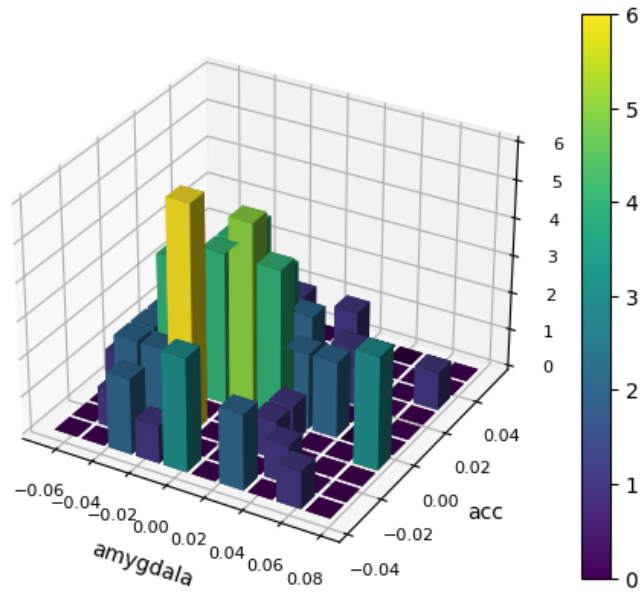
bin=15

From the above, we can tell either bin=8 (default) or 10 works. The more bins we have the more empty bins later in the histogram and the data points are more spread out over all bins so we can see the max count for each bin is less with increasing the number of bins. Like the last plot, when bin=15, it is too sparse.

When there are too few bins, many data points squeeze within a few bins making the plot very spiky.

The overall distribution is: modal is around (0,0), with several peaks around it. Data is skewed on the positive side of acc.

I have also tried the demo code and used bin=10 for each axis to form the histogram 2D plot.



2-dimensional histogram: bin=10 for each axis

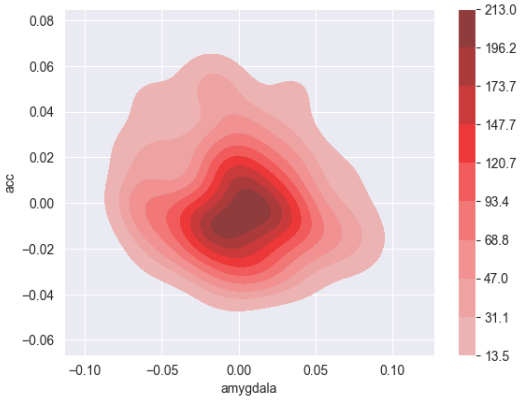
(c) Use kernel-density-estimation (KDE) to estimate the 2-dimensional density function of (amygdala, acc) (this means for this question, you can ignore the variable orientation). Set an appropriate kernel bandwidth $h > 0$.

Please show the two-dimensional KDE (e.g., two-dimensional heat-map, two-dimensional contour plot, etc.)

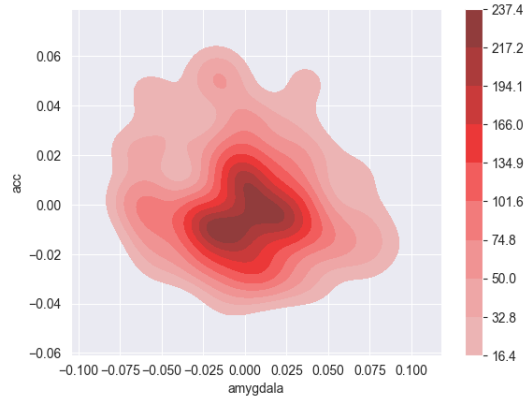
Please explain what you have observed: is the distribution unimodal or bi-modal? Are there any outliers?

Please explain based on the results, can you infer that the two variables (amygdala, acc) are likely to be independent or not?

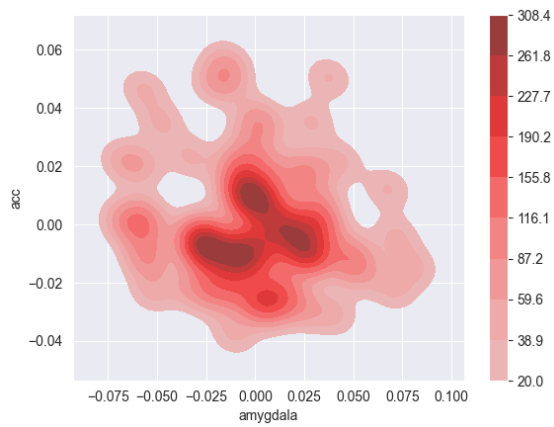
Answer:



bandwidth=default



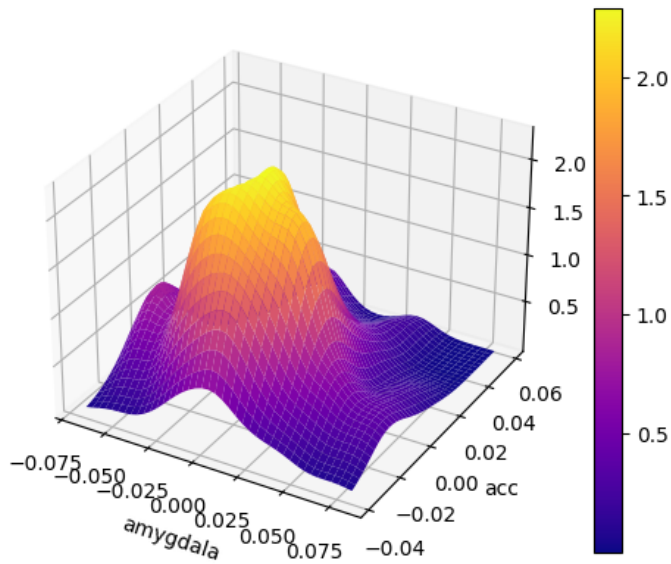
bandwidth=0.8



bandwidth=0.55

I prefer bandwidth=0.8 which is not that noisy but still keeps the shape of the distribution.

I have also tried to use bandwith=0.01 to form the KDE 2D plot.



2-dimensional KDE: bandwith=0.01

The interesting part is:

1. The shape of the KDE from my own code indeed matches the Seaborn contour plot with bandwith=0.55. We can see there are 3 very subtle tiny peaks located around the center. All of them are very close to each other.

2. Since the calculation of bandwidth is different, the good bandwidth for my own code is around 0.01. It is also very sensitive. When increased to 0.05, we lose all the details of the KDE surface shape. This 0.01 bandwidth also matches with the best bandwith=0.014 in the 1-dimensional KDE for amygdala and the best acc bandwidth=0.009 from Silverman's rule of thumb.

- **Based on my KDE plot, the distribution looks more like unimodal.**

Although when we adjust the bandwidth, we can see some noisy small peaks around the origin, there is no obvious bi-modal. Especially when I set the bandwidth=0.8, it is unimodal. So I will say the distribution is unimodal.

- **Based on my observation and calculation of outliers, there is a low chance of the presence of outliers.**

Firstly, I use Tukey's rule to test each dimension (amygdala and acc). Statistically, an outlier can be defined as:

$$Outlier_min < Q1 - 1.5 * IQR$$

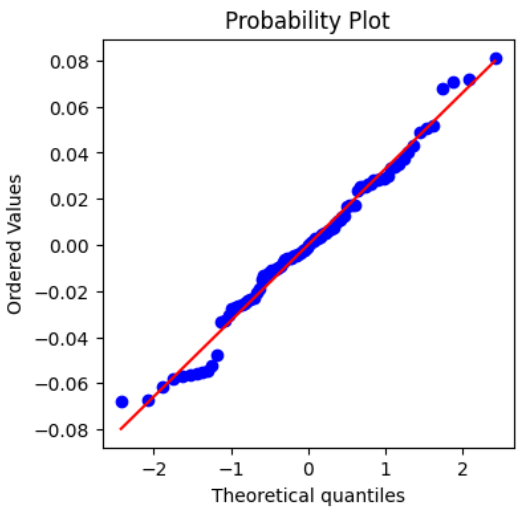
$$Outlier_max > Q3 + 1.5 * IQR$$

dimension	low outlier number	high outlier number
amygdala	0	0
acc	0	4

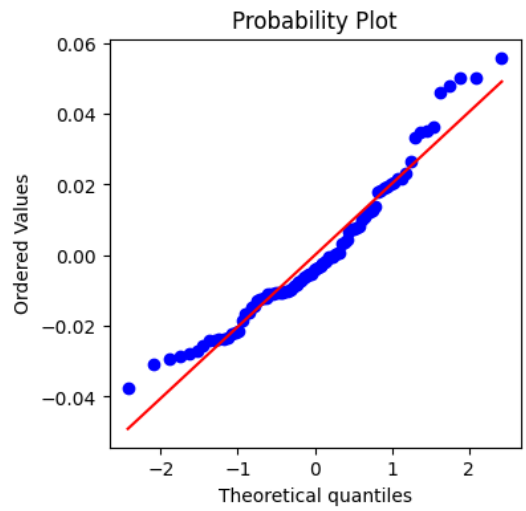
Interquartile range (IQR)=Q3-Q1, Q1=the 1st quartile , Q3=the 3rd quartile

For the dimension of amygdala, there isn't any data point above the upper bound or below the lower bound of Tukey's rule. For acc, there are only 4 data points bigger than the upper bound of Tukey's rule. However, these 4 data points' amygdala index is within the normal range.

Secondly, I have created the Q–Q plot here:



Q-Q plot for amygdala



Q-Q plot for acc

In the Q–Q plot, we can see for amygdala, the points are perfectly aligned with the identity line. For acc, the points on both sides are a bit off the identity line. But it is not too far away.

As a result, I will say there is not obviously the presence of outliers. If we want to investigate, we can try to dig deeper with those 4 points on the upper side of acc to see whether these points are legit or not.

- **The data of amygdala and acc are NOT independent**.

The independence definition in statistics is: "For two random variables X, Y, they are independent if and only if the joint distribution equals the product of two marginal distributions for any x,y in the corresponding support." This can be written as:

$$p(X = x, Y = y) = p(X = x) * p(Y = y)$$

The p here is probability. Based on my coding, I already have the pdf functions which produce the probability density value of the joint and marginal distributions. Normally, to get the probability from pdf function, we need to do integration. However, the integral of the joint pdf is the xy area which is canceled out by the product of the marginal pdf with their own integration on each axis. So we can directly use the pdf function we have from the Gaussian KED.

The steps are:

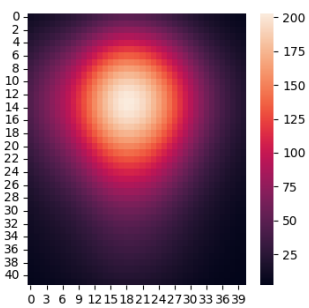
1. Created the mesh grids with 40 grids on each axis.

2. Create the plot for the product of the marginal pdf of both amygdala and acc over these 40*40 grids. Note: I use both bandwidth=0.6 here since this size of bandwidth is my favorite from part (a).

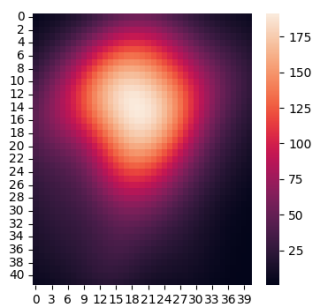
3. Create the plot for the joint pdf over these 40*40 grids. Note: I use both bandwidth=0.6 here too to match the marginal distribution.

4. Set a new variable: $\varepsilon = |p(X = x, Y = y) - p(X = x) * p(Y = y)|$. If amygdala and acc are independent, we will see this ε is generally equal to 0 all over these 40*40 grids.

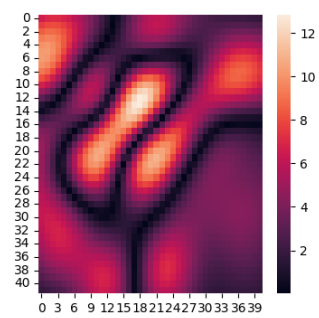
5. Create the plot for ε to check whether there is some high value.



p(X=x)*p(Y=y)



p(X=x,Y=y)



ε

From above we can see the ε values are definitely not 0 all over these 40*40 grids. So amygdala and acc are not independent.

(d) We will consider the variable orientation and consider conditional distributions. Please plot the estimated conditional distribution of amygdala conditioning on political orientation: $p(\text{amygdala}|\text{orientation} = c)$, $c = 2, \ldots, 5$, using KDE. Set an appropriate kernel bandwidth $h > 0$. Do the same for the volume of the acc: plot $p(\text{acc}|\text{orientation} = c)$, $c = 2, \ldots, 5$ using KDE. (Note that the conditional distribution can be understood as fitting a distribution for the data with the same orientation. Thus you should plot 8 one-dimensional distribution functions in total for this question.)

Now please explain based on the results, can you infer that the conditional distribution of amygdala and acc, respectively, are different from $c = 2, \ldots, 5$? This is a type of

scientific question one could infer from the data: Whether or not there is a difference between brain structure and political view.

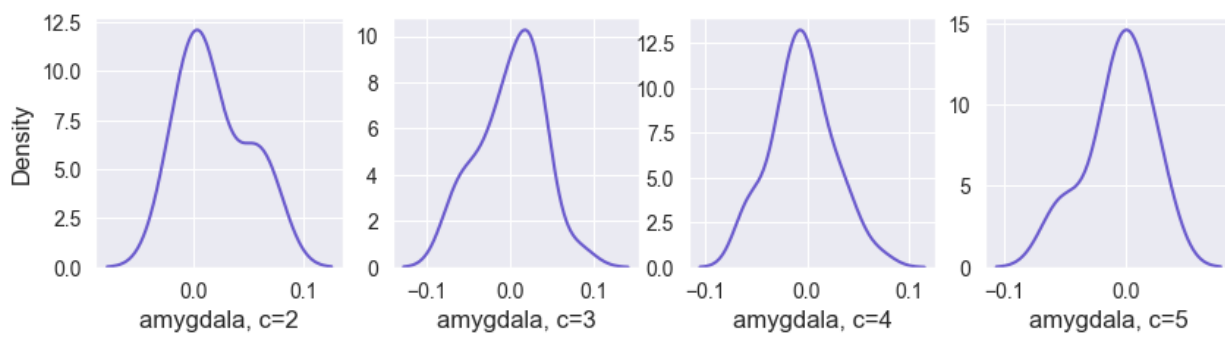
Now please also fill out the *conditional sample mean* for the two variables:

	$c = 2$	$c = 3$	$c = 4$	$c = 5$
amygdala	0.01906	0.00059	-0.00472	-0.00569
acc	-0.01477	0.00167	0.00131	0.00814

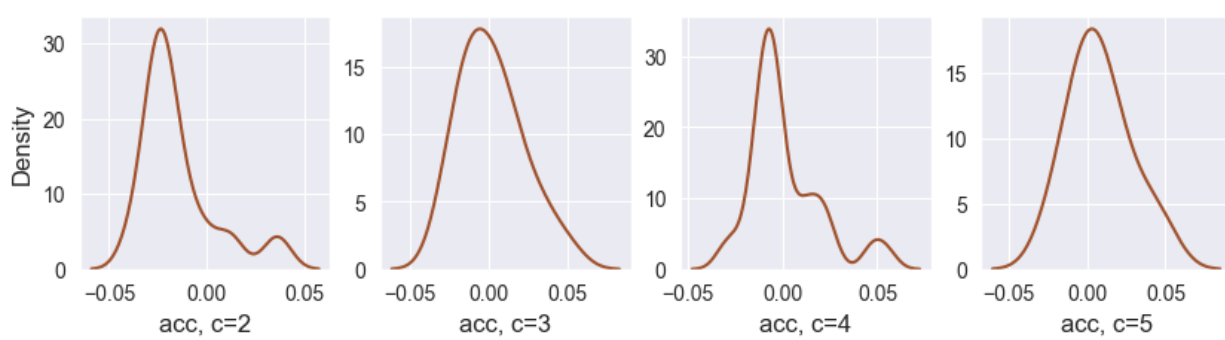
Remark: As you can see this exercise, you can extract so much more information from density estimation than simple summary statistics (e.g., the sample mean) in terms of explorable data analysis.

Answer:

Below are the KDE plots for the conditional distribution of amygdala conditioning on political orientation. I have tested a lot of times and the default bandwidth(1) from Seaborn is good enough. We can see the orientation of the skewness changes with the change of c.



Below are the KDE plots for the conditional distribution of acc conditioning on political orientation. I have tried to adjust the bandwidth and seems for c=2 and 4, bw=0.6 is good. For c=3 and 5, the default bandwidth is good enough. They are all right-skewed for c=2, 3, 4. But for c=2 and 4, the right tails are longer and noisier. With increasing of c, the modal is moving from -0.025 to 0.01.



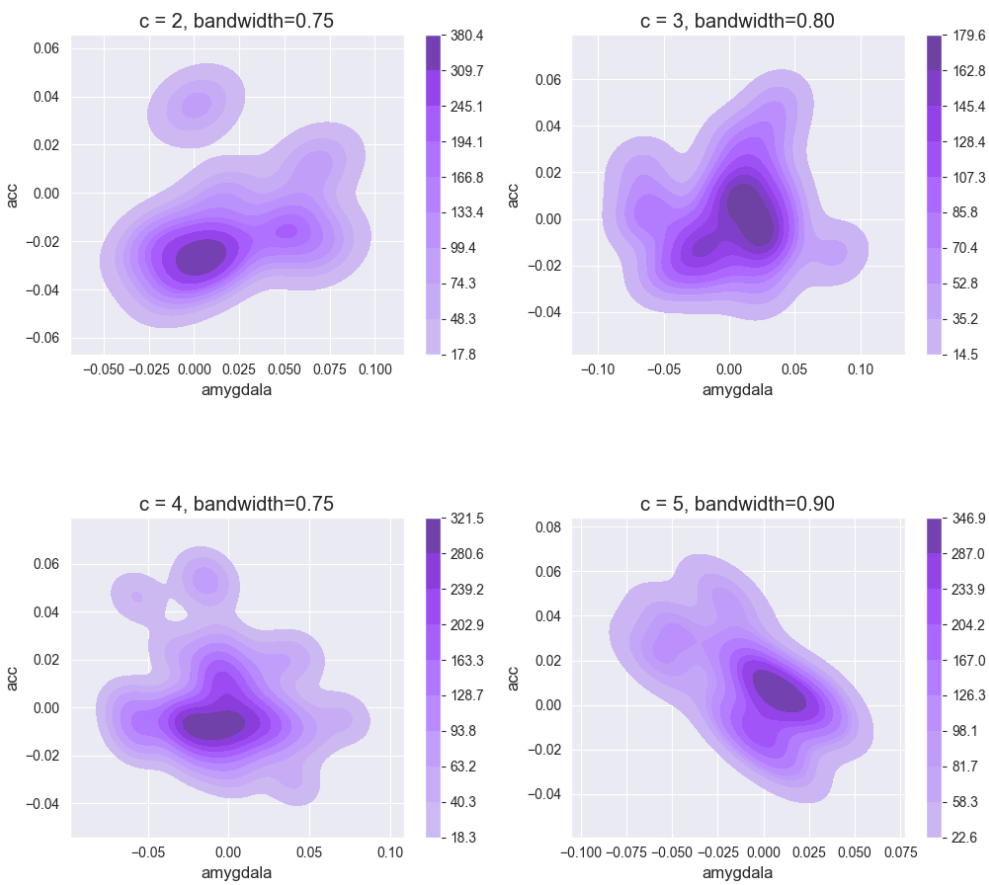
From the conditional sample mean, we can tell with increasing c, the size of the amygdala is smaller and smaller and the size of the acc is bigger and bigger. The KDE plots also look very different with different c values for both amygdala and acc. So I will say:

The conditional distributions of amygdala and acc, respectively, with different political orientation, seems different.

(e) Again we will consider the variable orientation. We will estimate the conditional *joint* distribution of the volume of the amygdala and acc, conditioning on a function of political orientation: $p(\text{amygdala}, \text{acc}|\text{orientation} = c)$, $c = 2, \ldots, 5$. You will use two-dimensional KDE to achieve the goal; et an appropriate kernel bandwidth $h > 0$. Please show the two-dimensional KDE (e.g., two-dimensional heat-map, two-dimensional contour plot, etc.).

Please explain based on the results, can you infer that the conditional distribution of two variables (amygdala, acc) are different from $c = 2, \ldots, 5$? This is a type of scientific question one could infer from the data: Whether or not there is a difference between brain structure and political view.

Answer:



The 2D KED plots for different orientations look very different. Seems there is a difference between brain structure and political view!

3. Implementing EM for MNIST dataset

Implement the EM algorithm for fitting a Gaussian mixture model for the MNIST handwritten digits dataset. For this question, we reduce the dataset to be only two cases, of digits "2" and "6" only. Thus, you will fit GMM with $C = 2$. Use the data file data.mat or data.dat. True label of the data are also provided in label.mat and label.dat.

The matrix images is of size 784-by-1990, i.e., there are 1990 images in total, and each column of the matrix corresponds to one image of size 28-by-28 pixels (the image is vectorized; the original image can be recovered by mapping the vector into a matrix).

First use PCA to reduce the dimensionality of the data before applying to EM. We will put all "6" and "2" digits together, to project the original data into 4-dimensional vectors.

Now implement EM algorithm for the projected data (with 4-dimensions).

(a) Write down detailed expression of the E-step and M-step in the EM algorithm (hint: when computing τ_k^i, you can drop the $(2\pi)^{n/2}$ factor from the numerator and denominator expression, since it will be canceled out; this can help avoid some numerical issues in computation).

Answer:

E-step:

In the E-step, our goal is to calculate τ_k^i which is the posterior probability of the ith sample belonging to label k. If we have m data points, and we assume there are k sub-Gaussian distributions, the count of τ_k^i should be m*k. The expectation over posterior distribution $f(\theta)$ is also produced here.

From question 1.3 (above), we already know τ_k^i:

$$\tau_k^i = \frac{p(x^i|z^i = k)p(z^i = k)}{\sum_{k'=1\ldots K} p(z^i = k', x^i)} = \frac{\pi_k \mathcal{N}(x^i|\mu_k, \Sigma_k)}{\sum_{k'=1\ldots K} \pi_{k'} \mathcal{N}(x^i|\mu_{k'}, \Sigma_{k'})}$$

$$= \frac{\pi_k * \frac{1}{|\Sigma_k|^{0.5}} * exp\left(-\frac{1}{2}(x^i - \mu_k)^T \Sigma_k^{-1}(x^i - \mu_k)\right)}{\sum_{k'=1\ldots K} \pi_{k'} * \frac{1}{|\Sigma_{k'}|^{0.5}} * exp\left(-\frac{1}{2}(x^i - \mu_{k'})^T \Sigma_{k'}^{-1}(x^i - \mu_{k'})\right)}$$

M-step:

Based on the value of τ_k^i and $f(\theta)$, in M-step, we can update the parameters (μ_k, Σ_k) of each sub-Gaussian distribution and their probability π_k.

$$
\begin{aligned}
f(\theta) &= E_{q(z^1,z^2...z^m)}\left[log\prod_{i=1}^{m}p(x^i,z^i|\theta)\right] \\
&= \sum_{i=1}^{m}E_{p(z^i|x^i,\theta^t)}\left[log\,p(x^i,z^i|\theta)\right] \\
&= \sum_{i=1}^{m}E_{p(z^i|x^i,\theta^t)}\left[log\,\pi_{z^i}\mathcal{N}(x^i|\mu_{z^i},\Sigma_{z^i})\right]
\end{aligned}
$$

For Gaussian pdf, we have:

$$
p(x|\mu,\Sigma) = \frac{1}{|\Sigma|^{1/2}(2\pi)^{n/2}}\,e^{-0.5(x-\mu)^T\Sigma^{-1}(x-\mu)}
$$

$$
\begin{aligned}
log(p) &= -log(|\Sigma|^{1/2}(2\pi)^{n/2}) - \frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu) \\
&= -\frac{1}{2}log(|\Sigma|) - \frac{n}{2}log(2\pi) - \frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)
\end{aligned}
$$

So for $f(\theta)$, we have:

$$
\begin{aligned}
f(\theta) &= \sum_{i=1}^{m}E_{p(z^i|x^i,\theta^t)}\left[log\pi_{z^i} - \frac{1}{2}log(|\Sigma_{z^i}|) - \frac{n}{2}log(2\pi) - \frac{1}{2}(x^i-\mu_{z^i})^T\Sigma_{z^i}^{-1}(x^i-\mu_{z^i})\right] \\
&= \sum_{i=1}^{m}\sum_{k=1}^{K}\tau_k^i\left[log\pi_k - \frac{1}{2}(x^i-\mu_k)^T\Sigma_k^{-1}(x^i-\mu_k) - \frac{1}{2}log(|\Sigma_k|) - \frac{n}{2}log(2\pi)\right]
\end{aligned}
$$

Form the Lagrangian relaxation of $f(\theta)$ with constraint $\sum_{k=1}^{K}\pi_k = 1$, we have:

$$
L = f(\theta) + \lambda(1 - \sum_{k=1}^{K}\pi_k)
$$

To get the optimal solution of $f(\theta)$, we set the 1st derivative of L with each variable (μ_k, Σ_k, π_k) equals to 0, we have:

$$
\pi_k = \frac{1}{m}\sum_{i=1}^{m}\tau_k^i,\ \mu_k = \frac{\sum_{i=1}^{m}\tau_k^i x^i}{\sum_{i=1}^{m}\tau_k^i},\ \Sigma_k = \frac{\sum_{i=1}^{m}\tau_k^i(x^i-\mu_k)(x^i-\mu_k)^T}{\sum_{i=1}^{m}\tau_k^i}
$$

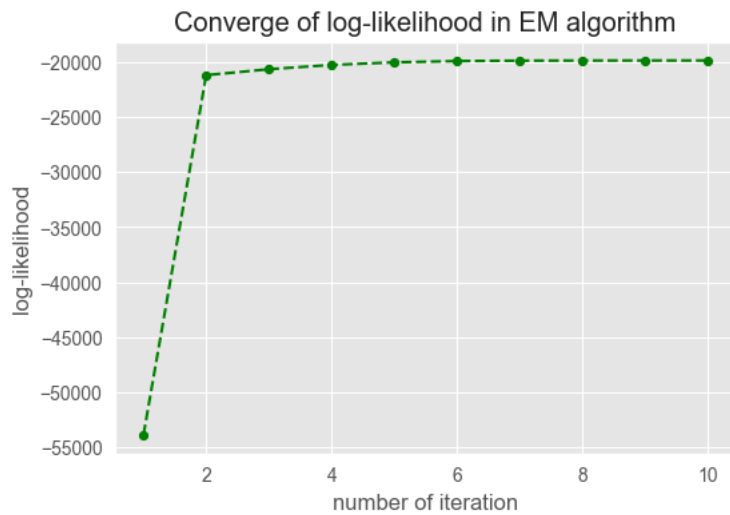
(b) Implement EM algorithm yourself. Use the following initialization

- initialization for mean: random Gaussian vector with zero mean

- initialization for covariance: generate two Gaussian random matrix of size n-by-n: S_1 and S_2, and initialize the covariance matrix for the two components are $\Sigma_1 = S_1 S_1^T + I_n$, and $\Sigma_2 = S_2 S_2^T + I_n$, where I_n is an identity matrix of size n-by-n.

Plot the log-likelihood function versus the number of iterations to show your algorithm is converging.

Answer:



From the plot, we can see the log-likelihood is converging during each iteration. The greatest jump is at iteration 2 which is really fast! I have tried to iterate 50 times and just ignore log-likelihood after the 10th iteration since the values nearly don't change after that.

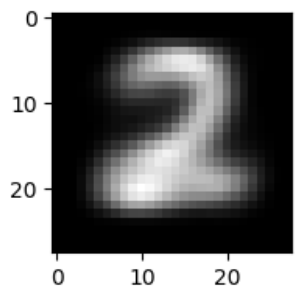
(c) Report, the fitted GMM model when EM has terminated in your algorithms as follows. Report the weights for each component, and the mean of each component, by mapping them back to the original space and reformat the vector to make them into 28-by-28 matrices and show images. Ideally, you should be able to see these means corresponds to some kind of "average" images. You can report the two 4-by-4 covariance matrices by visualizing their intensities (e.g., using a gray scaled image or heat map).

Answer:

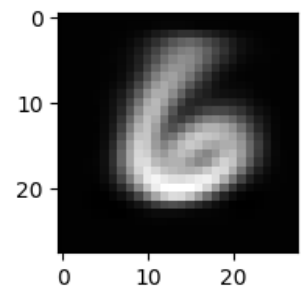
Note: Before PCA, I did preprocessing by having mean=0 and std=1 for each feature. Furthermore, I didn't do the whitening, so this will affect how I reverse the mean vector back. As a result, I don't need to dot product the diagonal eigenvalue matrix $\Lambda^{-1/2}$ to get my $\tilde{\mu}$ in the original space. So: $\tilde{\mu} = U_k \hat{\mu} * std + \gamma$ is enough! (γ: data mean vector in original space, std: data standard deviation vector for each feature (pixel) in original space, $\hat{\mu}$: sub-Gaussian mean in 4 PC space, $\tilde{\mu}$: sub-Gaussian mean in original space)

- weight for sub-Gaussian 1: 0.496
- weight for sub-Gaussian 2: 0.504
- mean for sub-Gaussian 1 is: [-5.46851005 0.68414922 0.94216152 0.79208489]
- mean for sub-Gaussian 2 is: [5.37520171 -0.67247569 -0.92608557 -0.77856967]

The above mean vectors are located in the 4-dimensional PC space. After mapping them back to the original space and reconstructing the vector, the mean images are shown below. We can easily tell one is digit 2 and the other is digit 6.

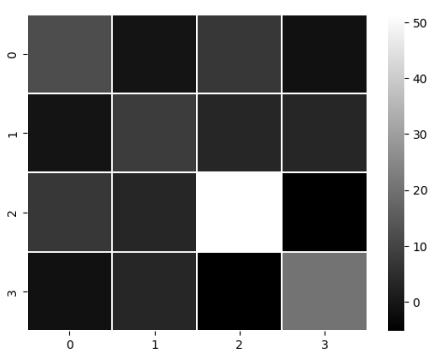


mean image for sub-Gaussian 1

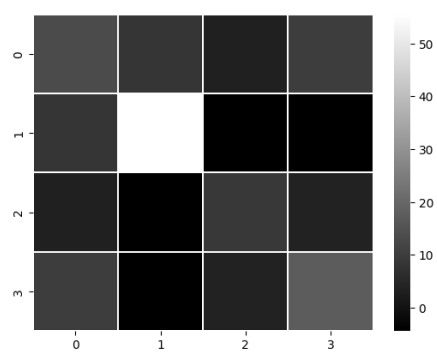


mean image for sub-Gaussian 2

The 4*4 covariance matrices in PC space are shown below:



covariance matrix for sub-Gaussian 1

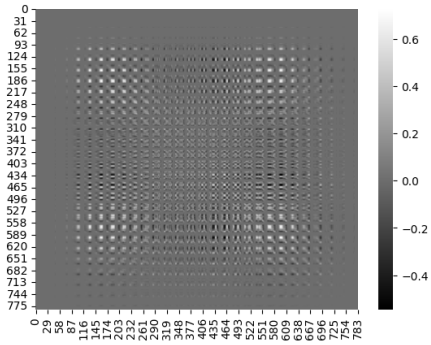


covariance matrix for sub-Gaussian 2

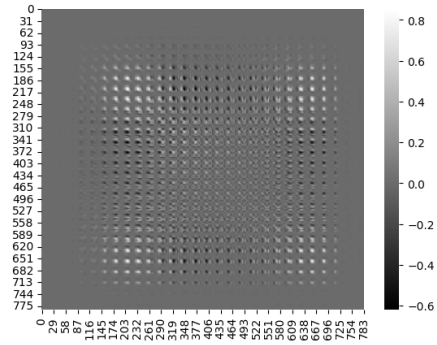
The 784*784 reconstructed covariance matrices in the original space are shown below (didn't add the mean back):

We can tell the 2 covariance matrices look very different.

(d) Use the τ_k^i to infer the labels of the images, and compare with the true labels. Report the mis-classification rate for digits "2" and "6" respectively. Perform K-means clustering with $K = 2$ (you may call a package or use the code from your previous homework). Find out the mis-classification rate for digits "2" and "6" respectively, and compare with GMM. Which one achieves the better performance?



784*784 covariance matrix (sub-Gaussian 1)



784*784 covariance matrix (sub-Gaussian 2)

Answer:

Note: there are conflicting explanations from different TAs for how to define the definition of "mis-classification rate". In order to show I truly understand, I just include both calculations below.

Approach 1:

For mis-classification rate, one interpretation is (take digit 2 as an example): "out of all of the true 2's, what percentage of them did we correctly classify as 2?"

If calculate as this interpretation, we have below mis-classification rates:

mis-classification rate		
	GMM	Kmeans
digit 2	6.01%	7.36%
digit 6	1.25%	3.24%

We can tell GMM is better than Kmeans for both 2 and 6!

Approach 2:

For mis-classification rate, according to ED post from TA, another interpretation is: "out of all the digits we classified as 2, how many were truly 2?" In the confusion matrix, it should be $1 - precision$.

I have computed the confusion matrices and show them below:

Confusion Matrix for GMM		
	GMM label for 2	GMM label for 6
True label for 2	970	62
True label for 6	12	946

So mis-classification rate for 2 is $\frac{12}{970+12} = 1.22\%$, mis-classification rate for 6 is $\frac{62}{946+62} = 6.15\%$.

Confusion Matrix for Kmeans		
	Kmeans label for 2	Kmeans label for 6
True label for 2	956	76
True label for 6	31	927

So mis-classification rate for 2 is $\frac{31}{956+31} = 3.14\%$, mis-classification rate for 6 is $\frac{76}{927+76} = 7.58\%$.

So comparing both GMM and Kmeans mis-classification rates, we have:

mis-classification rate		
	GMM	Kmeans
digit 2	1.22%	3.14%
digit 6	6.15%	7.58%

There are 1032 digit 2, 958 digit 6 in our dataset. No matter how we interpret the definition of "mis-classification rate", we can tell that GMM achieves better performance for both 2 and 6! Also, because there is some randomness in our algorithms, I have set seed=12 to ensure the reproduction of my results. In conclusion, GMM is indeed better than Kmeans!